Simon Says

ECE5725 Final Project

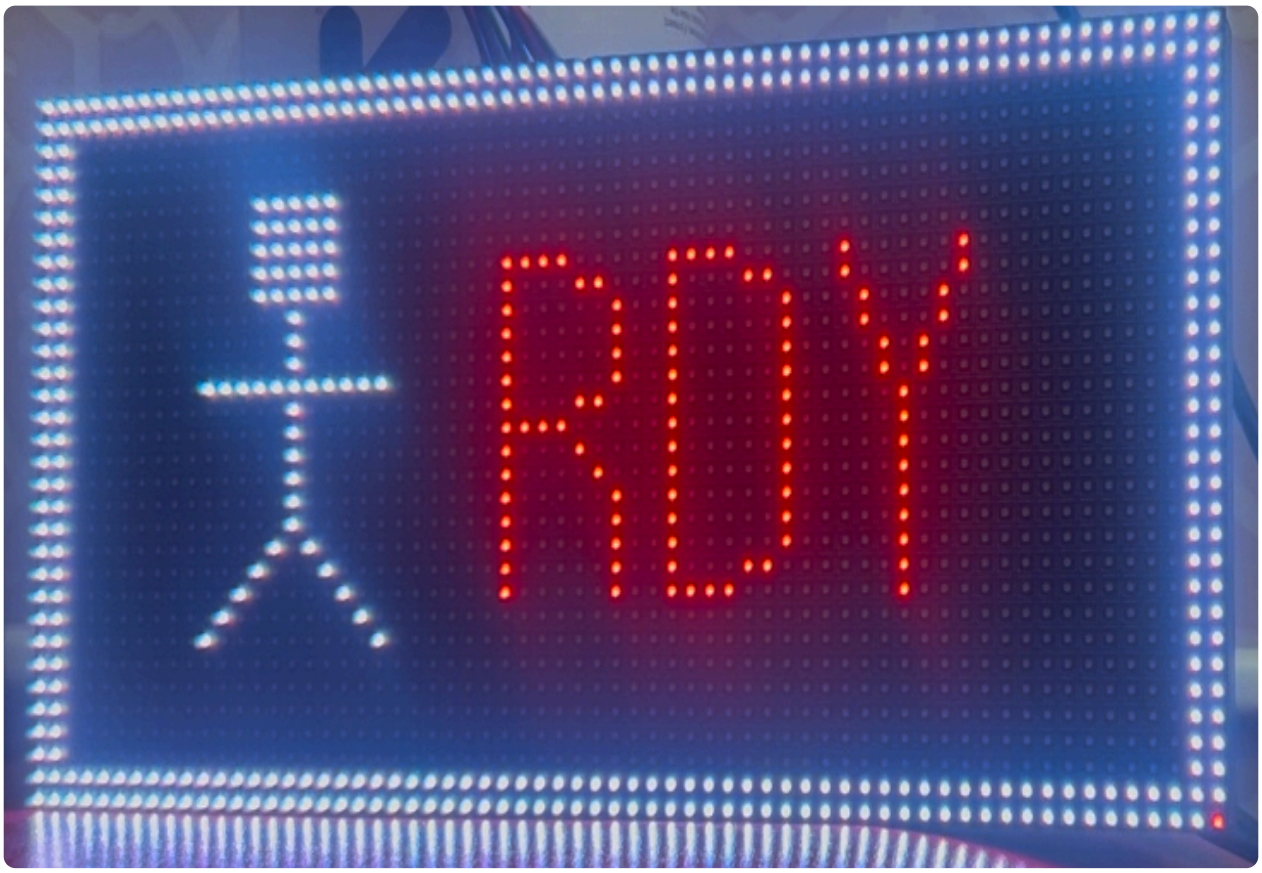
A Project By Henry Calderon and Elias Castro

Demonstration Video

Introduction

This project is based on the popular kids game Simon Says. This game involves a player getting told to do certain movements from the orchestrator “Simon” which in this case is the LED matrix screen. When the screen displays “SIMON SAYS” then the player must do the action shown on the screen in order to gain a point. If they do not then they will lose a point. Similarly, if the screen doesn’t display “SIMON SAYS” but the player performs the action then they will lose a point. Note that a point will not be gained here if Simon doesn't say anything and the player does nothing. They just keep the amount of points they have. One can only get points by doing what Simon says. In total there are 5 movements the player is asked to do. These moves include left and right arm out, left and right leg up, and finally arms up. The player's objective is to gain a set number of points so they can win. For each command you only have a few seconds to react. As you gain more points you have less time to react. Otherwise you can easily go on a losing streak and fall

back down to 0 points. To help keep track of the points, a scoreboard at the top right of the screen is displayed, and a stick figure corresponding to the movements is also there to help the player. In order to keep track of the players movements, a camera is used to track both wrists, feet, and head of the player. Using computer vision, we are able to render if the player does the correct movement by looking at the head landmark and seeing where the wrist landmarks are relative to it. Likewise for the feet, we can compare how high one ankle landmark is compared to the other.



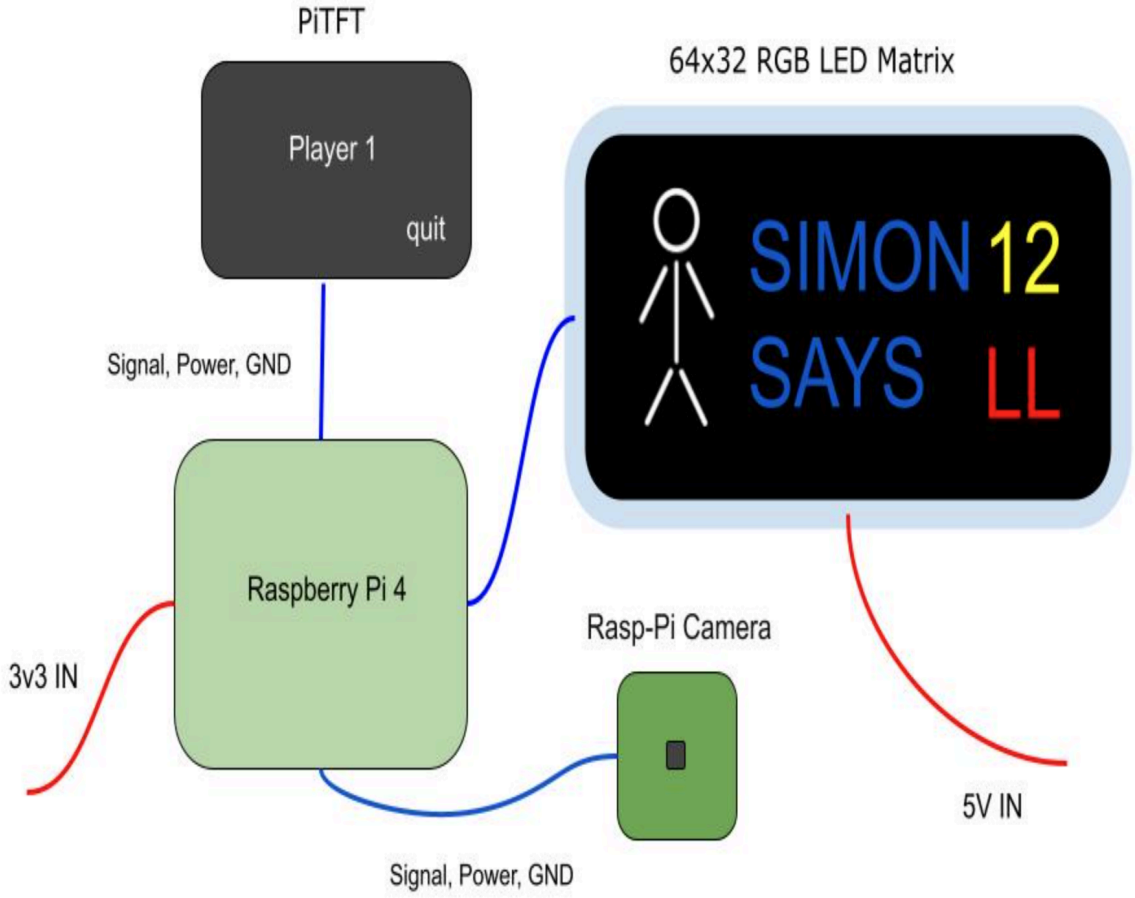
Project Objective:

- Use mediapipe to process landmarks around the body
- Display a 64x32 LED Matrix with Simon Says commands and using a stick figure emulating the actions
- Create game logic for Simon Says and use pigame to draw states on the piTFT
- Being able to process pictures from the picamera using CV2 and detecting 5 unique actions
- Have the fun and enjoyment of playing Simon Says on the Raspberry-Pi

---

Hardware Implemenation

Figure 1: Hardware Overview



The hardware we used for this project consisted of the Raspberry-Pi 4, Adafruit LED 64x32 Matrix, Raspberry-Pi Camera v2, and Adafruit PiTFT. To connect the PiTFT we plugged it in exactly the same way we did in previous labs. The LED matrix, requires us wiring 16 Rasp-Pi pins to it which fortunately the PiTFT didn't use any of them. The wiring of the pins for the matrix is shown in the table, referencing the pin layout provided by Adafruit. Lastly, to connect the camera, we just used the flexible ribbon cable that came with it. To power our system, the Raspberry-Pi provided the power to the PiTFT and camera, using the 3v3 power provided, and similary the LED matrix had a 5V power brick porvided that we connected to the back of the matrix. The back of the matrix had a Vin and ground connection that we connected the wire from the power brick too. To test if everything was working we simply just plugged in both power bricks, and turned on the LED matrix, PiTFT, and tested the camera modules which all worked as expected.

Software - LED Matrix - Henry

The LED Matrix was a small hurdle at first to understand how to get the LEDS to turn on. To first use the LED matrix, a Python module needs to be installed called rpi-rgb-led-matrix, to install it I looked up the module git, and then proceeded to download the shell script. After running the script, the module asks the user a few options to how they want to set it up. In my case, since I directly wired the LED matrix to the Pi, and didn't use for example an Adafruit Hat I just selected all the default options. Afterwards, when I first started coding, I was trying to draw simple lines and boxes to see if the matrix worked properly. However,

everytime I tried running my programs, there were always random errors in the lines and the matrix not working properly. Thinking that this was caused by a wiring issue, I rewired all the wires numerous times trying to figure out the issue. It wasn't till after I realized that the matrix requires a lot of global variables to be initialized to get it properly working. The most important variable was gpio_slowdown, which informed how fast the GPIO pins should be updated to turn on/off the LEDS. By default this is set to 1, which causes any drawing to flash and look horrific. Setting it to 4, made sure that the flickering stopped, and all images appeared properly. Afterwards setting up the other variables such as how many rows and columns they were was important, since the default module assumes that we are using a 32x32 matrix. After all the variables were property set, I tested the matrix by displaying some text and images which all appeared properly which then allowed me to proceed to coding our own frames for our game. Our Simon Says game has 5 actions as mentioned previously. The 5 actions are arms up, left arm out, right arm out, left leg up, and right leg up. For each action there was a Simon Says frame, and a non simon says frame. Meaning that in total we needed 10 frames to cover all the actions. Next, in order to showcase if you got it wrong or right I coded a correct frame which was a green checkmark, and a wrong screen which was a red x. Additionally, a frame for the starting screen was made where I displayed the letters "RDY" to indicate to the player that they should be ready to start. Lasly, I drew a trophy scene to indicate to the player that they won the game after they reach a certain score. In total 14 frames were created, all having different aspects. For the action and starting screen I drew a stick figure alongside it to showcase the movement that needs to be done. Furthermore, at the top right of the display a scoreboard was implemented to keep track of how many points the player is at. With a player getting a point each time they get it correct, and losing a point when they are wrong. In order to draw all these texts, figures, and scoreboard, I individually coded each pixel on the frames which took a long while to do. Reflecting on it, since my python isn't the strongest unlike my C and verilog I didn't know how to optimize my code. My partner, who was much more proficient, showcased to me how to properly use classes which could've saved me a lot of time, since I could've created functions to draw lines, and fixed images on the led matrix. That being said, the way I created my drawings was using the function SetPixel(z,y, R, G, B), where x and y are the coordinates relative to the origin which is at the top left corner of the matrix. RGB, is simply the RGB color code used for the pixel, which I varied to give a bit of life to the game. To create lines, I simply did for loops which was a bit of a hassle to use since I needed to create a lot more than I expected to draw so many letters and characters. However, in the end the images looked great and a showcase of some of the frames is shown below.

Software - Game Logic - Elias

When it comes to creating the game logic, the main question was how can we make the game fun but challenigng. The player can gain points only 1 way: by doing the correct move when Simon says to. To make sure the game doesn't go on for too long, we've set the percentage of time that Simon does say something to 65%. This gives the player a slight advantage to gain points and speed up the game. To increase the difficult as the game goes on, we set the reaction time smaller based on the number of points. To do this we set the base reaction time as 2.5 seconds. Then we do the following equation for the reaction time: change_time = 2.5 - 1.5 (points + 1 / win_points). Thus as the player gets closer to winning they must react faster as they will only have about 1 second of reaction time. For our five moves, we randomly choose an integer between 0 and 4. Each move corresponds to one integer. We then pass on the current move to the LED panel to display the appropriate screen. For every move, the camera will be capturing the players movements and append every move that the player does. To do this we make a set and add a unique move that was tracked by the camera within the given time window. With this set we can safely say that if Simon didn't say anything and this set is not empty then the player has clearly done a move and they lose a point. If the set is empty then they passed. If Simon did say something then we check 2 things: 1) The set only has a length of 1 as only 1 move should be registered and 2) the one move in the set should be equivalent

to Simon's commands. If both of these conditions are true then the player has clearly made the right move and gains a point. For registering moves and how it was done, please see the Testing section. Should the player win then they will have the option to restart. The player also has the option to go back to the starting screen midgame. Both options reset the entire game. Thus fields such as the change time and points are reset back to the original state.

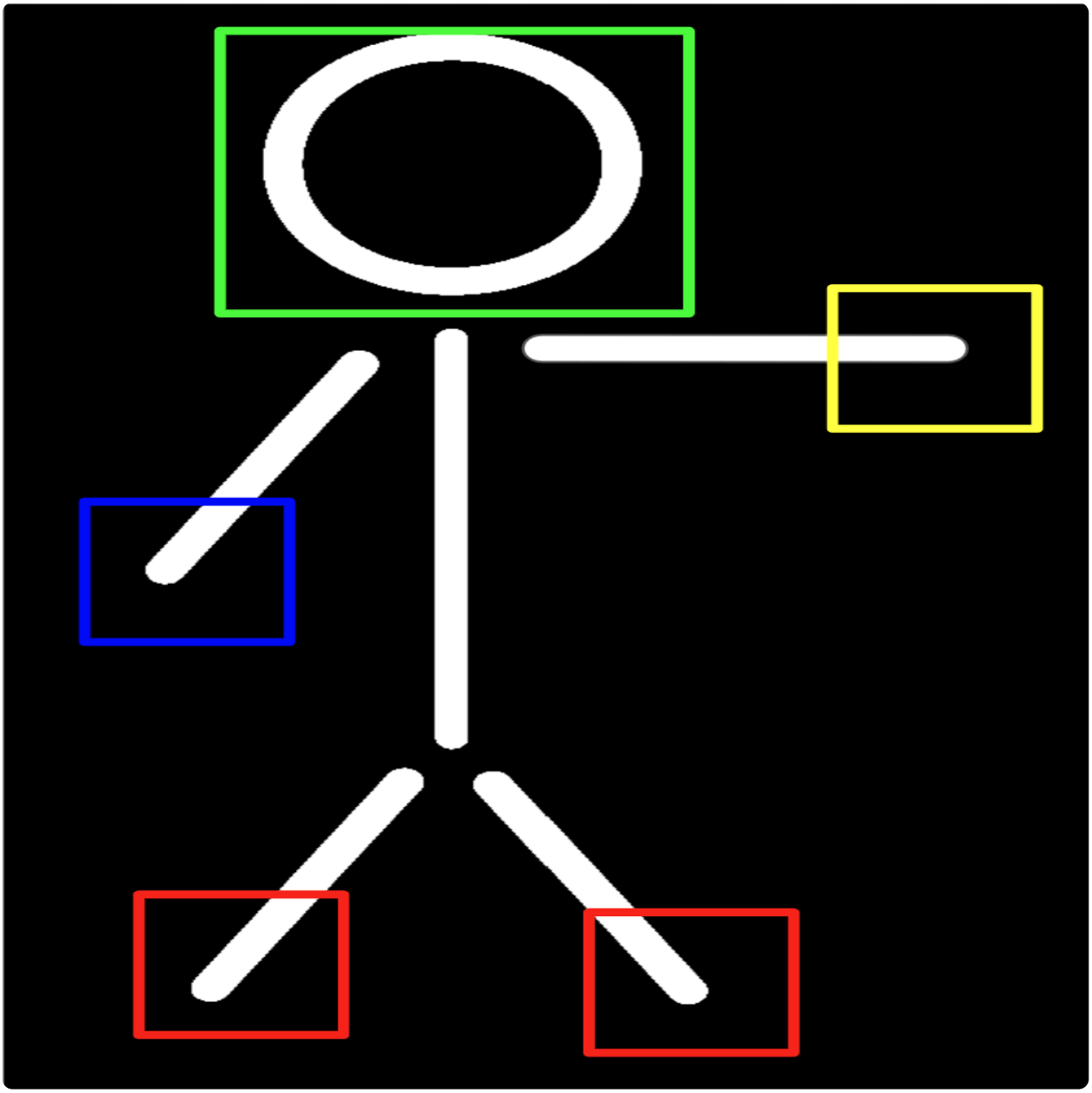
Testing

To test the camera, we ran the stream on a monitor using an HDMI cable. The first step for Simon detecting what we say was to keep track of certain landmarks of the body. To do this we used mediapipe to process landmarks. By landmarks we mean key features of the human body. There are five key features: The head, both wrists, and both ankles. At first we tried to use mediapipe's hand features but the camera had a hard time recognizing it. To bypass this, we used the wrist landmarks given by the pose landmarks. Since these were close to the hands, we could just treat them as such. For each landmark, we draw a box around it.

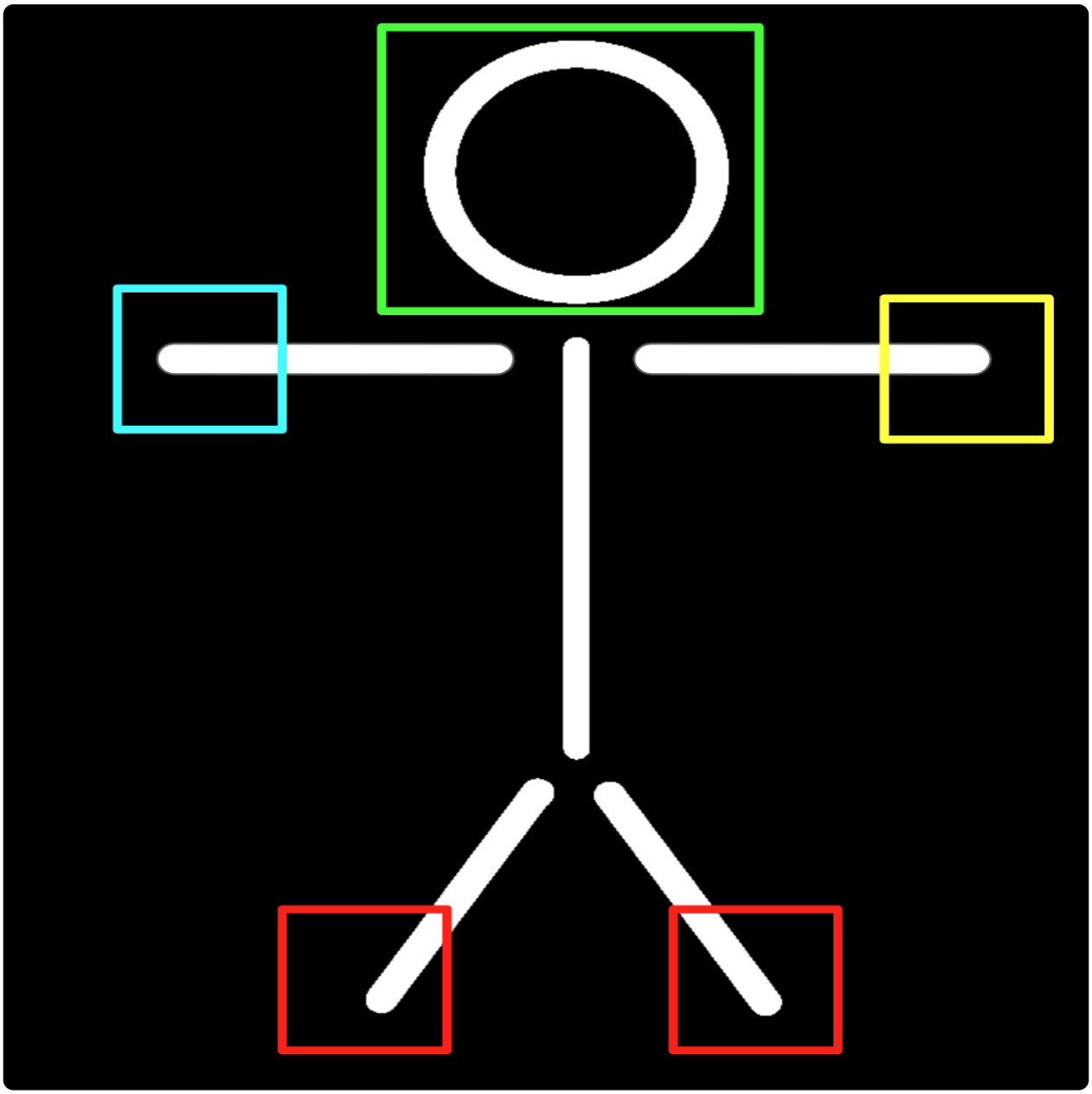
After recognizing all 5 landmarks, the next goal was to know when a move counts and when a move doesn't. We first start with the hands. There are three hand poses: right hand out, left hand out, and both hands up. For the first two, we set an absolute pixel distance. That is, the x coordinate of the wrist box should be atleast 175 pixels away from the x coordinate of the head box. From here we can deduce that the player is sticking out their hand. For putting hands up, we check if the bottom edge of the wrist box is above the top edge of the head box.

For feet, we did not use the head box as a reference. Instead, we based the y coordinates of the box of the feet to the other. The logic being that a player can only stand on one leg, else they are jumping or falling. Similar to the hands, we did an absolute pixel distance but instead we based off the y coordinate. Thus, for the left leg to be considered up it, the bottom edge of the box corresponding to the left ankle must be a certain distance above the bottom edge of the box corresponding to the right ankle.

Once we knew that the poses were being processed correctly, we then incorporated the camera processing in the simon says code. To test if the game logic was working correctly, we simply ran it. There were many minor errors such as points not being incremented correctly and getting confused between the left and right ankles. These errors were easy to fix.

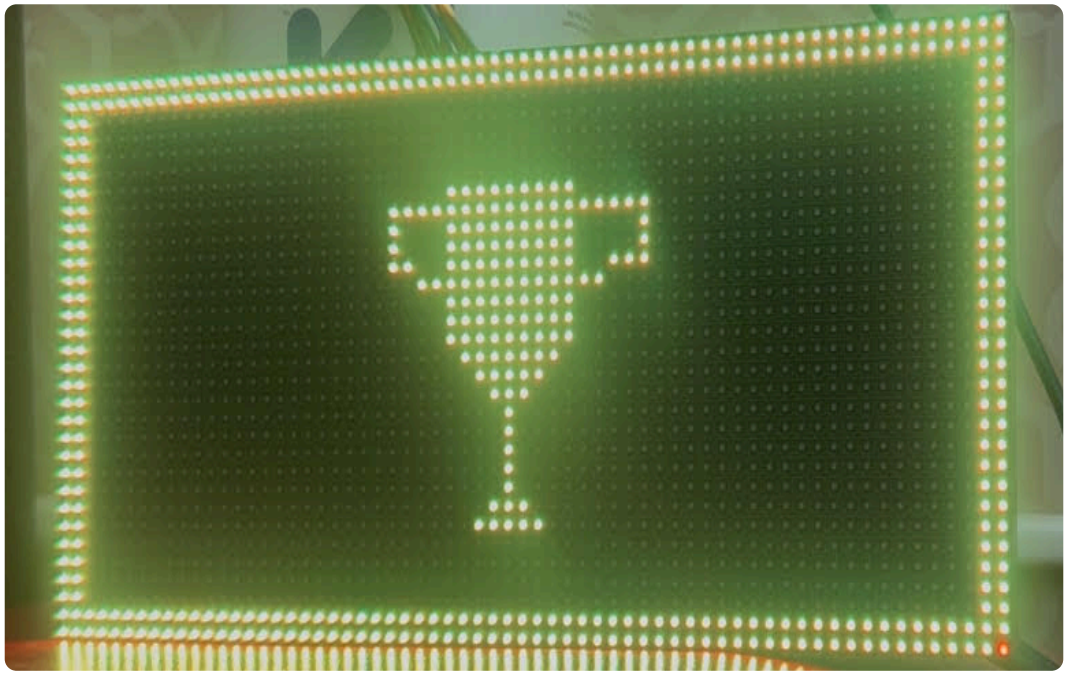


To give a better sense of how we processed moves, the box above shows the 5 landmarks. Each landmark has a color: Green for the head, blue for the wrists, and red for the ankles. In this image, the player is sticking their left hand out far enough to the point where it is registered as a move. Thus the color of the left wrist box changes color, allowing us to visualize when the move counts.



In this image, the player is sticking both of their hands out, thus at this frame the camera will count both moves. This shows that we can process multiple moves at once. Note that all valid moves will have the boxes change colors. This allowed us to eyeball what hte offset should be and what looked/felt most natural.

Result



As a final reflection, we see that we made an idea into reality using the piTFT. Neither of us have experience with the LED panel and the mediapipe interface. In short, to make the game possible we had to teach ourselves many tools and observe things such as the game flow by eye. The most important question to us was how can we make the experience best for the end user, that is how can we make the game as fun and easy to understand as possible. To prevent the game from being boring we played around with the reaction time and played a couple of times. Near the end, the reaction time is very fast and it is very possible to be sweating or atleast panting by the end!

We see that in the end we have a fully functional and addictive game. If there was more time, future work would include adding more moves to the game and possibly adding new land marks. In addition, we could add a speaker. Thus the player would have a voice and the LED panel telling them what move to do next. Another possible feature would be to add depth perception. Due to the absolute pixel calculation we used for relating the wrist landmarks to the head landmark, the player must stand at a certain distance away from the camera. A more flexible calculation would include depth, that is the closer the player is to the camera the larger the threshold would be. Finally, one could add a special "cheat code" move that would automatically win the game. Overall, the project has showed us that we can create our own custom games and gave us a taste of what hardware and software would be needed for creating such games.

Work Distribution

Henry

hcc54@cornell.edu

Worked on the LED matrix, and wiring

Elias Castro

ec798@cornell.edu

Interacted with the picamera, processed imagining, and designed the game

---

Parts List

- Raspberry Pi $35.00

- Raspberry Pi Camera V2 $25.00
- Raspberry Pi Tape - Provided in lab
- 64x32 RGB LED Matrix - 4mm pitch $39.95 (https://www.adafruit.com/product/2278)
- LEDs, Resistors and Wires - Provided in lab

Total: $99.95

References

PiCamera Document (https://picamera.readthedocs.io/)
MediaPipe (https://ai.google.dev/edge/mediapipe/solutions/guide)
open CV (https://pypi.org/project/opencv-python/)
Pigpio Library (http://abyz.co.uk/rpi/pigpio/)
R-Pi GPIO Document (https://sourceforge.net/p/raspberry-gpio-python/wiki/Home/)

Code Appendix

simon_says.py

```
# By Elias Castro (ec798)
import pygame,pigame
from pygame.locals import *
import os
import RPi.GPIO as GPIO
import cv2
import mediapipe as mp
from picamera2 import Picamera2
import random
import traceback
from time import sleep, time
import sys
sys.path.append('rpi-rgb-led-matrix/bindings/python/samples')

import testing_rgb as rgb
from rgbmatrix import RGBMatrix, RGBMatrixOptions


GPIO.setmode(GPIO.BCM)

#Colours
WHITE = (255,255,255)
commands = ['hands up', 'right leg up', 'left leg up', 'right hand out', 'left hand out']
os.putenv('SDL_VIDEODRV','fbcon')
os.putenv('SDL_FBDEV', '/dev/fb0')
os.putenv('SDL_MOUSEDRV','dummy')
os.putenv('SDL_MOUSEDEV','/dev/null')
os.putenv('DISPLAY','')

pygame.init()
pygame.mouse.set_visible(False)
pitft = pigame.PiTft() # create a pitft object for movig touch events
lcd = pygame.display.set_mode((320, 240))
lcd.fill((0,0,0))
pygame.display.update()

font_big = pygame.font.Font(None, 30)

playing = False

# Initialize Picamera2
picam2 = Picamera2()
config = picam2.create_preview_configuration(main={"size": (640, 480)})
picam2.set_controls({"FrameRate": 5})
picam2.configure(config)
```

```
picam2.start()

# Initialize Mediapipe
mp_pose = mp.solutions.pose
pose = mp_pose.Pose()

# Game logic
change = 2.5
win = False
points = 0
win_points = 15

# Keep track of simon says
command = None
registered = set()
simon_says = False

# run game
run_game = True
end_time = time()
change_time = time()

# Initialize square
simple_square = rgb.SimpleSquare()
simple_square.process()



def update_matrix(command, simon_says):
    if command == 'hands up':
      if simon_says:
        simple_square.armup_ss(points)
      else:
        simple_square.armup(points)
    elif command == 'right leg up':
      if simon_says:
        simple_square.rightleg_ss(points)
      else:
        simple_square.rightleg(points)
    elif command == 'left leg up':
      if simon_says:
          simple_square.leftleg_ss(points)
      else:
          simple_square.leftleg(points)
    elif command == 'right hand out':
      if simon_says:
          simple_square.rightarm_ss(points)
      else:
          simple_square.rightarm(points)
    else:
      if simon_says:
          simple_square.leftarm_SS(points)
      else:
```

```
            simple_square.leftarm(points)

    def process_camera():
      # list to keep track of moves
      results = []

      # Capture a frame using Picamera2
      frame = picam2.capture_array()

      # Convert the frame to RGB for MediaPipe
      rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

      # Process pose detection
      pose_results = pose.process(rgb_frame)

      # Initialize bounding boxes for head, hands, and legs
      head_bbox = None
      wrist_bboxes = []
      leg_bboxes = []

      # Extract pose landmarks to define bounding boxes
      top_head_y = None
      left_head_x = None
      right_head_x = None

      if pose_results.pose_landmarks:
          landmarks = pose_results.pose_landmarks.landmark

          # Use the nose landmark for head
          nose = landmarks[mp_pose.PoseLandmark.NOSE]
          head_bbox = (
              int(nose.x * rgb_frame.shape[1] - 30),
              int(nose.y * rgb_frame.shape[0] - 30),
              int(nose.x * rgb_frame.shape[1] + 30),
              int(nose.y * rgb_frame.shape[0] + 30),
          )

          # Use left and right ankle landmarks for legs
          left_ankle = landmarks[mp_pose.PoseLandmark.LEFT_ANKLE]
          right_ankle = landmarks[mp_pose.PoseLandmark.RIGHT_ANKLE]

          leg_bboxes.append((
              int(left_ankle.x * rgb_frame.shape[1] - 20),
              int(left_ankle.y * rgb_frame.shape[0] - 20),
              int(left_ankle.x * rgb_frame.shape[1] + 20),
              int(left_ankle.y * rgb_frame.shape[0] + 20),
          ))

          leg_bboxes.append((
              int(right_ankle.x * rgb_frame.shape[1] - 20),
              int(right_ankle.y * rgb_frame.shape[0] - 20),
              int(right_ankle.x * rgb_frame.shape[1] + 20),
              int(right_ankle.y * rgb_frame.shape[0] + 20),
```

```
        ))

        # Use left and right wrist landmarks
        left_wrist = landmarks[mp_pose.PoseLandmark.LEFT_WRIST]
        right_wrist = landmarks[mp_pose.PoseLandmark.RIGHT_WRIST]

        wrist_bboxes.append((
            int(left_wrist.x * rgb_frame.shape[1] - 20),
            int(left_wrist.y * rgb_frame.shape[0] - 20),
            int(left_wrist.x * rgb_frame.shape[1] + 20),
            int(left_wrist.y * rgb_frame.shape[0] + 20),
        ))

        wrist_bboxes.append((
            int(right_wrist.x * rgb_frame.shape[1] - 20),
            int(right_wrist.y * rgb_frame.shape[0] - 20),
            int(right_wrist.x * rgb_frame.shape[1] + 20),
            int(right_wrist.y * rgb_frame.shape[0] + 20),
        ))

    # Draw bounding boxes on the frame, colors useful for debugging
    # NOTE: Probably not best to put in final script
    if head_bbox:
        # Green for head
        top_head_y = head_bbox[1]
        left_head_x = head_bbox[0]
        right_head_x = head_bbox[2]

    above_count = 0
    for bbox in wrist_bboxes:
        # Blue for hands
        threshold = 175
        left_hand_x, _, right_hand_x, bottom_hand_y = bbox

        if top_head_y != None and bottom_hand_y < top_head_y:
          above_count += 1
        elif left_head_x != None and right_hand_x - left_head_x > threshold:
          results.append('left hand out')
        elif right_head_x != None and left_hand_x - right_head_x < -threshold:
          results.append('right hand out')

    if above_count == 2:
      results.append('hands up')

    # We only want to process if we have both legs not just 1
    if len(leg_bboxes) == 2:
        left_bbox = leg_bboxes[0]
        right_bbox = leg_bboxes[1]

        # Left up
        offset = 40
        if not (left_bbox[3] < right_bbox[3] + offset):
            results.append('right leg up')
```

```
      # Right up
      if not (right_bbox[3] < left_bbox[3] + offset):
          results.append('left leg up')


  return results

def display_win():
    lcd.fill((0,0,0))
    win_text = "Congragulations, you've won!"
    win_position = (160,80)
    text_surface = font_big.render('%s'%win_text, True, WHITE)
    rect = text_surface.get_rect(center=win_position)
    lcd.blit(text_surface, rect)

    play_again = "Play again"
    again_pos = (260,200)
    text_surface = font_big.render('%s'%play_again, True, WHITE)
    rect = text_surface.get_rect(center=again_pos)
    lcd.blit(text_surface, rect)

    pygame.display.update()

def update_touch(x = 0, y = 0):
  global playing
  global end_time
  global change_time

  simon_says = '' if random.random() < .35 else 'Simon Says:'
  index = random.randint(0,len(commands)-1)

  lcd.fill((0,0,0))

  # Pressed player 1 button
  if x > 120 and x < 220 and y > 120 and y < 180 and not playing:
    playing = True

    # Must do this so that game is in sync
    update_matrix(commands[index],simon_says)
    end_time = time()
    change_time = time()


  if not playing:
    touch_buttons = {'1 Player':(160,150), 'Welcome to Simon Says!':(160,80), 'Quit': (260,200)}
  else:
    # Playing da game
    touch_buttons = {'Back' : (260,200)}

    position = (160,100)
    text_surface = font_big.render('%s'%simon_says, True, WHITE)
    rect = text_surface.get_rect(center=position)
    lcd.blit(text_surface, rect)
```

```
    position = (160,120)
    command = 'Put your ' + commands[index]
    text_surface = font_big.render('%s'%command, True, WHITE)
    rect = text_surface.get_rect(center=position)
    lcd.blit(text_surface, rect)

    position = (160,140)
    command = 'Points: ' + str(points)
    text_surface = font_big.render('%s'%command, True, WHITE)
    rect = text_surface.get_rect(center=position)
    lcd.blit(text_surface, rect)

  for k,v in touch_buttons.items():
    text_surface = font_big.render('%s'%k, True, WHITE)
    rect = text_surface.get_rect(center=v)
    lcd.blit(text_surface, rect)

  pygame.display.update()

  if playing:
    return (simon_says == 'Simon Says:', commands[index])

# run initial display
update_touch()
end_time = time()
change_time = time()

try:
    while run_game:
        pitft.update()
        # Scan touchscreen events
        for event in pygame.event.get():
            if(event.type is MOUSEBUTTONDOWN):
              x,y = pygame.mouse.get_pos()
            elif(event.type is MOUSEBUTTONUP):
                x,y = pygame.mouse.get_pos()

                # We quit if it is in this rectangle
                if x >= 230 and y >= 180:
                  if not playing:
                    pygame.quit()
                    # Finally quit game
                    run_game = False
                  # Quit simon says
                  else:
                    # Reset logic
                    playing = not playing
                    win = not win if win else win
                    change = 2.5
                    points = 0
                  simple_square.clear_matrix()
```

```
        update_touch(x,y)

if win:
    display_win()
# Update after couple seconds
elif playing:
  if end_time - change_time > change:
    # Did simon say anything?
    if not simon_says:
      # Simon didnt say it so no moves should be registed, take away a point
      if len(registered) > 0:
        if points > 0:
            points -= 1
        simple_square.wrong()
      else:
      #   points += 1
        simple_square.correct()
    # Only 1 move and that move should be simons command
    else:
      if command in registered and len(registered) == 1:
        points += 1
        simple_square.correct()
      else:
          if points > 0:
            points -= 1
          simple_square.wrong()
    sleep(.75)


    if points >= win_points:
      win = True
      simple_square.winner()
      continue

    simon_says, command = update_touch()
    update_matrix(command, simon_says)
    # Create new set
    registered = set()
    change_time = time()
    change = 1.75 - ((points+1) / win_points)
  else: # Window for getting it correct
    # Keep track of moves
    moves = process_camera()
    for move in moves:
        # add move, since it is set we do not need to worry abt duplicates
        registered.add(move)
# Decrease by .1 to give less reaction time
else:
  simple_square.waiting()


sleep(0.1)
end_time = time()
```

```
    except Exception as e:
        # Open a text file in write mode to log the error
        with open("error_log.txt", "w") as error_file:
            # Write the error message
            error_file.write("An error occurred:\n")
            error_file.write(str(e) + "\n\n")

            # Write the full traceback for debugging
            error_file.write("Traceback details:\n")
            traceback.print_exc(file=error_file)

    # clean up after everything is done
    del(pitft)
    # picam2.stop()
    cv2.destroyAllWindows()


}
```

testing_rgb.py

```
# By Henry Calderon (hcc54)
from samplebase import SampleBase


class SimpleSquare(SampleBase):
    def __init__(self, *args, **kwargs):
        super(SimpleSquare, self).__init__(*args, **kwargs)

    def waiting(self):
        offset_canvas = self.matrix.CreateFrameCanvas()
        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 174, 198, 207)
            offset_canvas.SetPixel(x, 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 174, 198, 207)
            offset_canvas.SetPixel(1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

        #Spine
        for y in range(9):
            offset_canvas.SetPixel(15, y+10, 174, 198, 207)

        #ARM
        for x in range(11):
            offset_canvas.SetPixel(x+10, 13, 174, 198, 207)

        #Leg
        for x in range(6):
            #right leg
            offset_canvas.SetPixel(x+15, x+19, 174, 198, 207)
            #left leg
            offset_canvas.SetPixel(offset_canvas.height - 17 - x, x+19, 174, 198, 207)



        #head
        for y in range(5):
            for x in range(5):
                offset_canvas.SetPixel(x+13, y+5, 174, 198, 207)

        #ready R
        for y in range(15):
            offset_canvas.SetPixel(27, y+8, 255, 0, 0)
            # D
```

```
    offset_canvas.SetPixel(36, y+8, 255, 0, 0)

for x in range(2):
    offset_canvas.SetPixel(x+31, 9, 255, 0, 0)
    offset_canvas.SetPixel(x+31, 14, 255, 0, 0)
    offset_canvas.SetPixel(x+29, 8, 255, 0, 0)
    offset_canvas.SetPixel(x+29, 15, 255, 0, 0)
    # D
    offset_canvas.SetPixel(x+38, 8, 255, 0, 0)
    offset_canvas.SetPixel(x+40, 9, 255, 0, 0)
    offset_canvas.SetPixel(x+38, 22, 255, 0, 0)
    offset_canvas.SetPixel(x+40, 21, 255, 0, 0)

for y in range(4):
    offset_canvas.SetPixel(33, y+10, 255, 0, 0)

for y in range(5):
    offset_canvas.SetPixel(33, y+18, 255, 0, 0)

for y in range(11):
    offset_canvas.SetPixel(42, y+10, 255, 0, 0)

for y in range(9):
    offset_canvas.SetPixel(48, y+14, 255, 0, 0)

for y in range(2):
    offset_canvas.SetPixel(45, y+8, 255, 0, 0)
    offset_canvas.SetPixel(46, y+10, 255, 0, 0)
    offset_canvas.SetPixel(47, y+12, 255, 0, 0)
    offset_canvas.SetPixel(51, y+8, 255, 0, 0)
    offset_canvas.SetPixel(50, y+10, 255, 0, 0)
    offset_canvas.SetPixel(49, y+12, 255, 0, 0)


# for x in range(4):
#     #right leg
#     offset_canvas.SetPixel(x+47, x+8, 174, 198, 207)
#     #left leg
#     offset_canvas.SetPixel(offset_canvas.height +21 - x, x+8, 174, 198, 207)

offset_canvas.SetPixel(32, 17, 255, 0, 0)
offset_canvas.SetPixel(31, 16, 255, 0, 0)
offset_canvas.SetPixel(28, 8, 255, 0, 0)
offset_canvas.SetPixel(28, 15, 255, 0, 0)
offset_canvas.SetPixel(37, 8, 255, 0, 0)
offset_canvas.SetPixel(37, 22, 255, 0, 0)

# ready D


offset_canvas = self.matrix.SwapOnVSync(offset_canvas)
```

```
def clear_matrix(self):
    offset_canvas = self.matrix.CreateFrameCanvas()
    offset_canvas.Clear()

#  Left Leg Simon Says
def leftleg_ss(self, z):
    offset_canvas = self.matrix.CreateFrameCanvas()

    for x in range(0, offset_canvas.width):
        offset_canvas.SetPixel(x, 0, 174, 198, 207)
        offset_canvas.SetPixel(x, 1, 174, 198, 207)
        offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
        offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

    for y in range(0, offset_canvas.height):
        offset_canvas.SetPixel(0, y, 174, 198, 207)
        offset_canvas.SetPixel(1, y, 174, 198, 207)
        offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
        offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

    #Spine
    for y in range(9):
        offset_canvas.SetPixel(12, y+10, 174, 198, 207)

    #Legs
    for x in range(6):
        #right leg
        offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
        #left leg
        #offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

    #Arms
    for x in range(4):
        #right arm
        offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
        #left arm
        offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)


    #head
    for y in range(5):
        for x in range(5):
            offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

    #Left leg
    for x in range(6):
        offset_canvas.SetPixel(x+7, 19, 174, 198, 207)


    for y in range(7):
```

```
        # S I M O N S A Y S
        # I, M, O, N, A
        #I
        offset_canvas.SetPixel(30, y+5, 0, 255, 255)
        # M
        offset_canvas.SetPixel(32, y+5, 0, 255, 255)
        offset_canvas.SetPixel(36, y+5, 0, 255, 255)
        # O
        offset_canvas.SetPixel(38, y+5, 0, 255, 255)
        offset_canvas.SetPixel(42, y+5, 0, 255, 255)
        # N
        offset_canvas.SetPixel(44, y+5, 0, 255, 255)
        offset_canvas.SetPixel(48, y+5, 0, 255, 255)
        # A
        offset_canvas.SetPixel(30, y+15, 0, 255, 255)
        offset_canvas.SetPixel(34, y+15, 0, 255, 255)

        # Command Letters
        # L
        offset_canvas.SetPixel(50, y+15, 255, 0, 0)

        offset_canvas.SetPixel(56, y+15, 255, 0, 0)

    for x in range(5):
        #s
        offset_canvas.SetPixel(x+24, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 8, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 11, 0, 255, 255)

        offset_canvas.SetPixel(x+24, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 21, 0, 255, 255)

        offset_canvas.SetPixel(x+42, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 21, 0, 255, 255)

        #M
        offset_canvas.SetPixel(x+32, 5, 0, 255, 255)
        #O
        offset_canvas.SetPixel(x+38, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+38, 11, 0, 255, 255)
        #N
        offset_canvas.SetPixel(x+44, 5, 0, 255, 255)
        #A
        offset_canvas.SetPixel(x+30, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+30, 18, 0, 255, 255)
        #Y
        offset_canvas.SetPixel(x+36, 18, 0, 255, 255)

        # Command Letters
        # L
        offset_canvas.SetPixel(x+50, 21, 255, 0, 0)
```

```
            offset_canvas.SetPixel(x+56, 21, 255, 0, 0)

        for y in range(4):
            #S
            offset_canvas.SetPixel(28, y+8, 0, 255, 255)
            offset_canvas.SetPixel(24, y+5, 0, 255, 255)

            offset_canvas.SetPixel(24, y+15, 0, 255, 255)
            offset_canvas.SetPixel(28, y+18, 0, 255, 255)

            offset_canvas.SetPixel(42, y+15, 0, 255, 255)
            offset_canvas.SetPixel(46, y+18, 0, 255, 255)

            # M
            offset_canvas.SetPixel(34, y+5, 0, 255, 255)
            # Y

            offset_canvas.SetPixel(36, y+15, 0, 255, 255)
            offset_canvas.SetPixel(40, y+15, 0, 255, 255)
            offset_canvas.SetPixel(38, y+18, 0, 255, 255)

            # Command Letters
        self.scoreboard(z,offset_canvas)
        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    #  Left Leg NOT SS
    def leftleg(self, z):
        offset_canvas = self.matrix.CreateFrameCanvas()

        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 174, 198, 207)
            offset_canvas.SetPixel(x, 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 174, 198, 207)
            offset_canvas.SetPixel(1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

        #Spine
        for y in range(9):
            offset_canvas.SetPixel(12, y+10, 174, 198, 207)

        #Legs
        for x in range(6):
            #right leg
            offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
            #left leg
            #offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

        #Arms
```

```
        for x in range(4):
            #right arm
            offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
            #left arm
            offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)

        #head
        for y in range(5):
            for x in range(5):
                offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

        #Left leg
        for x in range(6):
            offset_canvas.SetPixel(x+7, 19, 174, 198, 207)


        for y in range(7):
            # Command Letters
            # L
            offset_canvas.SetPixel(50, y+15, 255, 0, 0)

            offset_canvas.SetPixel(56, y+15, 255, 0, 0)

        for x in range(5):
            # Command Letters
            # L
            offset_canvas.SetPixel(x+50, 21, 255, 0, 0)
            offset_canvas.SetPixel(x+56, 21, 255, 0, 0)

        self.scoreboard(z,offset_canvas)
        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    #  Left Arm Simon Says
    def leftarm_SS(self, z):
        offset_canvas = self.matrix.CreateFrameCanvas()

        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 174, 198, 207)
            offset_canvas.SetPixel(x, 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 174, 198, 207)
            offset_canvas.SetPixel(1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

        #Spine
        for y in range(9):
            offset_canvas.SetPixel(12, y+10, 174, 198, 207)

        #Legs
```

```
for x in range(6):
    #right leg
    offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
    #left leg
    offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

#Arms
for x in range(4):
    #right arm
    offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
    #left arm
    #offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)


#head
for y in range(5):
    for x in range(5):
        offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

#Left Arm
for x in range(5):
    offset_canvas.SetPixel(x+7, 14, 174, 198, 207)


for y in range(7):
    # S I M O N S A Y S
    # I, M, O, N, A
    #I
    offset_canvas.SetPixel(30, y+5, 0, 255, 255)
    # M
    offset_canvas.SetPixel(32, y+5, 0, 255, 255)
    offset_canvas.SetPixel(36, y+5, 0, 255, 255)
    # O
    offset_canvas.SetPixel(38, y+5, 0, 255, 255)
    offset_canvas.SetPixel(42, y+5, 0, 255, 255)
    # N
    offset_canvas.SetPixel(44, y+5, 0, 255, 255)
    offset_canvas.SetPixel(48, y+5, 0, 255, 255)
    # A
    offset_canvas.SetPixel(30, y+15, 0, 255, 255)
    offset_canvas.SetPixel(34, y+15, 0, 255, 255)

    # Command Letters
    # L
    offset_canvas.SetPixel(50, y+15, 255, 0, 0)

    #A
    offset_canvas.SetPixel(56, y+15, 255, 0, 0)
    offset_canvas.SetPixel(60, y+15, 255, 0, 0)


for x in range(5):
    #s
```

```
        offset_canvas.SetPixel(x+24, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 8, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 11, 0, 255, 255)

        offset_canvas.SetPixel(x+24, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 21, 0, 255, 255)

        offset_canvas.SetPixel(x+42, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 21, 0, 255, 255)

        #M
        offset_canvas.SetPixel(x+32, 5, 0, 255, 255)
        #O
        offset_canvas.SetPixel(x+38, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+38, 11, 0, 255, 255)
        #N
        offset_canvas.SetPixel(x+44, 5, 0, 255, 255)
        #A
        offset_canvas.SetPixel(x+30, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+30, 18, 0, 255, 255)
        #Y
        offset_canvas.SetPixel(x+36, 18, 0, 255, 255)

        # Command Letters
        # L
        offset_canvas.SetPixel(x+50, 21, 255, 0, 0)
        #A
        offset_canvas.SetPixel(x+56, 15, 255, 0, 0)
        offset_canvas.SetPixel(x+56, 18, 255, 0, 0)

    for y in range(4):
        #S
        offset_canvas.SetPixel(28, y+8, 0, 255, 255)
        offset_canvas.SetPixel(24, y+5, 0, 255, 255)

        offset_canvas.SetPixel(24, y+15, 0, 255, 255)
        offset_canvas.SetPixel(28, y+18, 0, 255, 255)

        offset_canvas.SetPixel(42, y+15, 0, 255, 255)
        offset_canvas.SetPixel(46, y+18, 0, 255, 255)

        # M
        offset_canvas.SetPixel(34, y+5, 0, 255, 255)
        # Y

        offset_canvas.SetPixel(36, y+15, 0, 255, 255)
        offset_canvas.SetPixel(40, y+15, 0, 255, 255)
        offset_canvas.SetPixel(38, y+18, 0, 255, 255)

        # Command Letters
    self.scoreboard(z,offset_canvas)
```

```
    offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

#  Left Arm not SS
def leftarm(self, z):
    offset_canvas = self.matrix.CreateFrameCanvas()

    for x in range(0, offset_canvas.width):
        offset_canvas.SetPixel(x, 0, 174, 198, 207)
        offset_canvas.SetPixel(x, 1, 174, 198, 207)
        offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
        offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

    for y in range(0, offset_canvas.height):
        offset_canvas.SetPixel(0, y, 174, 198, 207)
        offset_canvas.SetPixel(1, y, 174, 198, 207)
        offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
        offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

    #Spine
    for y in range(9):
        offset_canvas.SetPixel(12, y+10, 174, 198, 207)

    #Legs
    for x in range(6):
        #right leg
        offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
        #left leg
        offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

    #Arms
    for x in range(4):
        #right arm
        offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
        #left arm
        #offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)


    #head
    for y in range(5):
        for x in range(5):
            offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

    #Left Arm
    for x in range(5):
        offset_canvas.SetPixel(x+7, 14, 174, 198, 207)


    for y in range(7):
        # Command Letters
        # L
        offset_canvas.SetPixel(50, y+15, 255, 0, 0)

        #A
```

```
            offset_canvas.SetPixel(56, y+15, 255, 0, 0)
            offset_canvas.SetPixel(60, y+15, 255, 0, 0)


        for x in range(5):
            # Command Letters
            # L
            offset_canvas.SetPixel(x+50, 21, 255, 0, 0)
            #A
            offset_canvas.SetPixel(x+56, 15, 255, 0, 0)
            offset_canvas.SetPixel(x+56, 18, 255, 0, 0)

        self.scoreboard(z,offset_canvas)
        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    #  Right Leg Simon Says
    def rightleg_ss(self, z):
        offset_canvas = self.matrix.CreateFrameCanvas()

        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 174, 198, 207)
            offset_canvas.SetPixel(x, 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 174, 198, 207)
            offset_canvas.SetPixel(1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

        #Spine
        for y in range(9):
            offset_canvas.SetPixel(12, y+10, 174, 198, 207)

        #Legs
        for x in range(6):
            #right leg
            #offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
            #left leg
            offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

        #Arms
        for x in range(4):
            #right arm
            offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
            #left arm
            offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)
            offset_canvas.SetPixel(x+51, x+18, 255, 0, 0)


        #head
        for y in range(5):
```

```
        for x in range(5):
            offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

    #Right leg
    for x in range(6):
        offset_canvas.SetPixel(x+13, 19, 174, 198, 207)


    for y in range(7):
        # S I M O N S A Y S
        # I, M, O, N, A
        #I
        offset_canvas.SetPixel(30, y+5, 0, 255, 255)
        # M
        offset_canvas.SetPixel(32, y+5, 0, 255, 255)
        offset_canvas.SetPixel(36, y+5, 0, 255, 255)
        # O
        offset_canvas.SetPixel(38, y+5, 0, 255, 255)
        offset_canvas.SetPixel(42, y+5, 0, 255, 255)
        # N
        offset_canvas.SetPixel(44, y+5, 0, 255, 255)
        offset_canvas.SetPixel(48, y+5, 0, 255, 255)
        # A
        offset_canvas.SetPixel(30, y+15, 0, 255, 255)
        offset_canvas.SetPixel(34, y+15, 0, 255, 255)

        # Command Letters
        # R
        offset_canvas.SetPixel(50, y+15, 255, 0, 0)
        # L
        offset_canvas.SetPixel(56, y+15, 255, 0, 0)

    for x in range(5):
        #s
        offset_canvas.SetPixel(x+24, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 8, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 11, 0, 255, 255)

        offset_canvas.SetPixel(x+24, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 21, 0, 255, 255)

        offset_canvas.SetPixel(x+42, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 21, 0, 255, 255)

        #M
        offset_canvas.SetPixel(x+32, 5, 0, 255, 255)
        #O
        offset_canvas.SetPixel(x+38, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+38, 11, 0, 255, 255)
        #N
        offset_canvas.SetPixel(x+44, 5, 0, 255, 255)
```

```
            #A
            offset_canvas.SetPixel(x+30, 15, 0, 255, 255)
            offset_canvas.SetPixel(x+30, 18, 0, 255, 255)
            #Y
            offset_canvas.SetPixel(x+36, 18, 0, 255, 255)

            # Command Letters
            # R
            offset_canvas.SetPixel(x+50, 15, 255, 0, 0)
            offset_canvas.SetPixel(x+50, 18, 255, 0, 0)
            # L
            offset_canvas.SetPixel(x+56, 21, 255, 0, 0)

        for y in range(4):
            #S
            offset_canvas.SetPixel(28, y+8, 0, 255, 255)
            offset_canvas.SetPixel(24, y+5, 0, 255, 255)

            offset_canvas.SetPixel(24, y+15, 0, 255, 255)
            offset_canvas.SetPixel(28, y+18, 0, 255, 255)

            offset_canvas.SetPixel(42, y+15, 0, 255, 255)
            offset_canvas.SetPixel(46, y+18, 0, 255, 255)

            # M
            offset_canvas.SetPixel(34, y+5, 0, 255, 255)
            # Y

            offset_canvas.SetPixel(36, y+15, 0, 255, 255)
            offset_canvas.SetPixel(40, y+15, 0, 255, 255)
            offset_canvas.SetPixel(38, y+18, 0, 255, 255)

            # Command Letters
            # R
            offset_canvas.SetPixel(54, y+15, 255, 0, 0)
        self.scoreboard(z,offset_canvas)
        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    #  Right Leg not SS
    def rightleg(self, z):
        offset_canvas = self.matrix.CreateFrameCanvas()

        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 174, 198, 207)
            offset_canvas.SetPixel(x, 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 174, 198, 207)
            offset_canvas.SetPixel(1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)
```

```
#Spine
for y in range(9):
    offset_canvas.SetPixel(12, y+10, 174, 198, 207)

#Legs
for x in range(6):
    #right leg
    #offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
    #left leg
    offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

#Arms
for x in range(4):
    #right arm
    offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
    #left arm
    offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)
    # R diagnol
    offset_canvas.SetPixel(x+51, x+18, 255, 0, 0)


#head
for y in range(5):
    for x in range(5):
        offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

#Right leg
for x in range(6):
    offset_canvas.SetPixel(x+13, 19, 174, 198, 207)


for y in range(7):
    # Command Letters
    # R
    offset_canvas.SetPixel(50, y+15, 255, 0, 0)
    # L
    offset_canvas.SetPixel(56, y+15, 255, 0, 0)

for x in range(5):
    # Command Letters
    # R
    offset_canvas.SetPixel(x+50, 15, 255, 0, 0)
    offset_canvas.SetPixel(x+50, 18, 255, 0, 0)
    # L
    offset_canvas.SetPixel(x+56, 21, 255, 0, 0)

for y in range(4):
    # Command Letters
    # R
    offset_canvas.SetPixel(54, y+15, 255, 0, 0)

self.scoreboard(z,offset_canvas)
```

```
    offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

# Right Arm Simon Says
def rightarm_ss(self, z):
    offset_canvas = self.matrix.CreateFrameCanvas()

    for x in range(0, offset_canvas.width):
        offset_canvas.SetPixel(x, 0, 174, 198, 207)
        offset_canvas.SetPixel(x, 1, 174, 198, 207)
        offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
        offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

    for y in range(0, offset_canvas.height):
        offset_canvas.SetPixel(0, y, 174, 198, 207)
        offset_canvas.SetPixel(1, y, 174, 198, 207)
        offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
        offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

    #Spine
    for y in range(9):
        offset_canvas.SetPixel(12, y+10, 174, 198, 207)

    #Legs
    for x in range(6):
        #right leg
        offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
        #left leg
        offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

    #Arms
    for x in range(4):
        #right arm
        #offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
        #left arm
        offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)
        # R diagnol
        offset_canvas.SetPixel(x+51, x+18, 255, 0, 0)


    #head
    for y in range(5):
        for x in range(5):
            offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

    #Right Arm
    for x in range(5):
        offset_canvas.SetPixel(x+13, 14, 174, 198, 207)


    for y in range(7):
        # S I M O N S A Y S
        # I, M, O, N, A
        #I
```

```
        offset_canvas.SetPixel(30, y+5, 0, 255, 255)
        # M
        offset_canvas.SetPixel(32, y+5, 0, 255, 255)
        offset_canvas.SetPixel(36, y+5, 0, 255, 255)
        # O
        offset_canvas.SetPixel(38, y+5, 0, 255, 255)
        offset_canvas.SetPixel(42, y+5, 0, 255, 255)
        # N
        offset_canvas.SetPixel(44, y+5, 0, 255, 255)
        offset_canvas.SetPixel(48, y+5, 0, 255, 255)
        # A
        offset_canvas.SetPixel(30, y+15, 0, 255, 255)
        offset_canvas.SetPixel(34, y+15, 0, 255, 255)

        # Command Letters
        # R
        offset_canvas.SetPixel(50, y+15, 255, 0, 0)

        #A
        offset_canvas.SetPixel(56, y+15, 255, 0, 0)
        offset_canvas.SetPixel(60, y+15, 255, 0, 0)


    for x in range(5):
        #s
        offset_canvas.SetPixel(x+24, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 8, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 11, 0, 255, 255)

        offset_canvas.SetPixel(x+24, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 21, 0, 255, 255)

        offset_canvas.SetPixel(x+42, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 21, 0, 255, 255)

        #M
        offset_canvas.SetPixel(x+32, 5, 0, 255, 255)
        #O
        offset_canvas.SetPixel(x+38, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+38, 11, 0, 255, 255)
        #N
        offset_canvas.SetPixel(x+44, 5, 0, 255, 255)
        #A
        offset_canvas.SetPixel(x+30, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+30, 18, 0, 255, 255)
        #Y
        offset_canvas.SetPixel(x+36, 18, 0, 255, 255)

        # Command Letters
        # R
        offset_canvas.SetPixel(x+50, 15, 255, 0, 0)
```

```
            offset_canvas.SetPixel(x+50, 18, 255, 0, 0)
            #A
            offset_canvas.SetPixel(x+56, 15, 255, 0, 0)
            offset_canvas.SetPixel(x+56, 18, 255, 0, 0)

        for y in range(4):
            #S
            offset_canvas.SetPixel(28, y+8, 0, 255, 255)
            offset_canvas.SetPixel(24, y+5, 0, 255, 255)

            offset_canvas.SetPixel(24, y+15, 0, 255, 255)
            offset_canvas.SetPixel(28, y+18, 0, 255, 255)

            offset_canvas.SetPixel(42, y+15, 0, 255, 255)
            offset_canvas.SetPixel(46, y+18, 0, 255, 255)

            # M
            offset_canvas.SetPixel(34, y+5, 0, 255, 255)
            # Y

            offset_canvas.SetPixel(36, y+15, 0, 255, 255)
            offset_canvas.SetPixel(40, y+15, 0, 255, 255)
            offset_canvas.SetPixel(38, y+18, 0, 255, 255)

            # Command Letters
            # R
            offset_canvas.SetPixel(54, y+15, 255, 0, 0)
        self.scoreboard(z,offset_canvas)
        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    # Right Arm not SS
    def rightarm(self, z):
        offset_canvas = self.matrix.CreateFrameCanvas()

        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 174, 198, 207)
            offset_canvas.SetPixel(x, 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 174, 198, 207)
            offset_canvas.SetPixel(1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

        #Spine
        for y in range(9):
            offset_canvas.SetPixel(12, y+10, 174, 198, 207)

        #Legs
        for x in range(6):
            #right leg
```

```
            offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
            #left leg
            offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

        #Arms
        for x in range(4):
            #right arm
            #offset_canvas.SetPixel(x+12, x+13, 174, 198, 207)
            #left arm
            offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+13, 174, 198, 207)
            # R diagnol
            offset_canvas.SetPixel(x+51, x+18, 255, 0, 0)


        #head
        for y in range(5):
            for x in range(5):
                offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

        #Right Arm
        for x in range(5):
            offset_canvas.SetPixel(x+13, 14, 174, 198, 207)


        for y in range(7):
            # Command Letters
            # R
            offset_canvas.SetPixel(50, y+15, 255, 0, 0)

            #A
            offset_canvas.SetPixel(56, y+15, 255, 0, 0)
            offset_canvas.SetPixel(60, y+15, 255, 0, 0)


        for x in range(5):
            # Command Letters
            # R
            offset_canvas.SetPixel(x+50, 15, 255, 0, 0)
            offset_canvas.SetPixel(x+50, 18, 255, 0, 0)
            #A
            offset_canvas.SetPixel(x+56, 15, 255, 0, 0)
            offset_canvas.SetPixel(x+56, 18, 255, 0, 0)

        for y in range(4):
            # Command Letters
            # R
            offset_canvas.SetPixel(54, y+15, 255, 0, 0)
        self.scoreboard(z,offset_canvas)
        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    # Arm Up Simon Says
    def armup_ss(self, z):
        offset_canvas = self.matrix.CreateFrameCanvas()
```

```
for x in range(0, offset_canvas.width):
    offset_canvas.SetPixel(x, 0, 174, 198, 207)
    offset_canvas.SetPixel(x, 1, 174, 198, 207)
    offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
    offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

for y in range(0, offset_canvas.height):
    offset_canvas.SetPixel(0, y, 174, 198, 207)
    offset_canvas.SetPixel(1, y, 174, 198, 207)
    offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
    offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

#Spine
for y in range(9):
    offset_canvas.SetPixel(12, y+10, 174, 198, 207)

#Legs
for x in range(6):
    #right leg
    offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
    #left leg
    offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

#Arms
for x in range(4):
    #Left arm
    offset_canvas.SetPixel(x+8, x+11, 174, 198, 207)
    #Right arm
    offset_canvas.SetPixel(offset_canvas.height - 16 - x, x+11, 174, 198, 207)


#head
for y in range(5):
    for x in range(5):
        offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

for y in range(7):
    # S I M O N S A Y S
    # I, M, O, N, A
    #I
    offset_canvas.SetPixel(30, y+5, 0, 255, 255)
    # M
    offset_canvas.SetPixel(32, y+5, 0, 255, 255)
    offset_canvas.SetPixel(36, y+5, 0, 255, 255)
    # O
    offset_canvas.SetPixel(38, y+5, 0, 255, 255)
    offset_canvas.SetPixel(42, y+5, 0, 255, 255)
    # N
    offset_canvas.SetPixel(44, y+5, 0, 255, 255)
    offset_canvas.SetPixel(48, y+5, 0, 255, 255)
    # A
    offset_canvas.SetPixel(30, y+15, 0, 255, 255)
```

```
        offset_canvas.SetPixel(34, y+15, 0, 255, 255)

        # Command Letters
        # A
        offset_canvas.SetPixel(50, y+15, 255, 0, 0)
        offset_canvas.SetPixel(54, y+15, 255, 0, 0)

        #U
        offset_canvas.SetPixel(56, y+15, 255, 0, 0)
        offset_canvas.SetPixel(60, y+15, 255, 0, 0)


    for x in range(5):
        #s
        offset_canvas.SetPixel(x+24, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 8, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 11, 0, 255, 255)

        offset_canvas.SetPixel(x+24, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+24, 21, 0, 255, 255)

        offset_canvas.SetPixel(x+42, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 18, 0, 255, 255)
        offset_canvas.SetPixel(x+42, 21, 0, 255, 255)

        #M
        offset_canvas.SetPixel(x+32, 5, 0, 255, 255)
        #O
        offset_canvas.SetPixel(x+38, 5, 0, 255, 255)
        offset_canvas.SetPixel(x+38, 11, 0, 255, 255)
        #N
        offset_canvas.SetPixel(x+44, 5, 0, 255, 255)
        #A
        offset_canvas.SetPixel(x+30, 15, 0, 255, 255)
        offset_canvas.SetPixel(x+30, 18, 0, 255, 255)
        #Y
        offset_canvas.SetPixel(x+36, 18, 0, 255, 255)

        # Command Letters
        # A
        offset_canvas.SetPixel(x+50, 15, 255, 0, 0)
        offset_canvas.SetPixel(x+50, 18, 255, 0, 0)
        # U
        offset_canvas.SetPixel(x+56, 21, 255, 0, 0)

    for y in range(4):
        #S
        offset_canvas.SetPixel(28, y+8, 0, 255, 255)
        offset_canvas.SetPixel(24, y+5, 0, 255, 255)

        offset_canvas.SetPixel(24, y+15, 0, 255, 255)
        offset_canvas.SetPixel(28, y+18, 0, 255, 255)
```

```
            offset_canvas.SetPixel(42, y+15, 0, 255, 255)
            offset_canvas.SetPixel(46, y+18, 0, 255, 255)

            # M
            offset_canvas.SetPixel(34, y+5, 0, 255, 255)
            # Y

            offset_canvas.SetPixel(36, y+15, 0, 255, 255)
            offset_canvas.SetPixel(40, y+15, 0, 255, 255)
            offset_canvas.SetPixel(38, y+18, 0, 255, 255)

            # Command Letters
        self.scoreboard(z,offset_canvas)
        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    # Arm Up Not SS
    def armup(self, z):
        offset_canvas = self.matrix.CreateFrameCanvas()
        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 174, 198, 207)
            offset_canvas.SetPixel(x, 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 174, 198, 207)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 174, 198, 207)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 174, 198, 207)
            offset_canvas.SetPixel(1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 174, 198, 207)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 174, 198, 207)

        #Spine
        for y in range(9):
            offset_canvas.SetPixel(12, y+10, 174, 198, 207)

        #Legs
        for x in range(6):
            #right leg
            offset_canvas.SetPixel(x+12, x+19, 174, 198, 207)
            #left leg
            offset_canvas.SetPixel(offset_canvas.height - 20 - x, x+19, 174, 198, 207)

        #Arms
        for x in range(4):
            #Left arm
            offset_canvas.SetPixel(x+8, x+11, 174, 198, 207)
            #Right arm
            offset_canvas.SetPixel(offset_canvas.height - 16 - x, x+11, 174, 198, 207)



        #head
        for y in range(5):
            for x in range(5):
```

```
            offset_canvas.SetPixel(x+10, y+7, 174, 198, 207)

    for y in range(7):
        # Command Letters
        # A
        offset_canvas.SetPixel(50, y+15, 255, 0, 0)
        offset_canvas.SetPixel(54, y+15, 255, 0, 0)

        #U
        offset_canvas.SetPixel(56, y+15, 255, 0, 0)
        offset_canvas.SetPixel(60, y+15, 255, 0, 0)


    for x in range(5):
        # Command Letters
        # A
        offset_canvas.SetPixel(x+50, 15, 255, 0, 0)
        offset_canvas.SetPixel(x+50, 18, 255, 0, 0)
        # U
        offset_canvas.SetPixel(x+56, 21, 255, 0, 0)
    self.scoreboard(z,offset_canvas)
    offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

# Correct check mark
def correct(self):
    offset_canvas = self.matrix.CreateFrameCanvas()
    for x in range(0, offset_canvas.width):
        offset_canvas.SetPixel(x, 0, 0, 255, 0)
        offset_canvas.SetPixel(x, 1, 0, 255, 0)
        offset_canvas.SetPixel(x, offset_canvas.height - 1, 0, 255, 0)
        offset_canvas.SetPixel(x, offset_canvas.height - 2, 0, 255, 0)

    for y in range(0, offset_canvas.height):
        offset_canvas.SetPixel(0, y, 0, 255, 0)
        offset_canvas.SetPixel(1, y, 0, 255, 0)
        offset_canvas.SetPixel(offset_canvas.width - 1, y, 0, 255, 0)
        offset_canvas.SetPixel(offset_canvas.width - 2, y, 0, 255, 0)


    for y in range(2):
        for x in range(7):
            offset_canvas.SetPixel(x+21, y+14, 0, 255, 0)
            y = y+1

    for y in range(2):
        for x in range(15):
            offset_canvas.SetPixel(offset_canvas.height + 10 - x, y+6, 0, 255, 0)
            y = y+1

    offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

# Wrong X mark
def wrong(self):
```

```
        offset_canvas = self.matrix.CreateFrameCanvas()

        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 255, 0, 0)
            offset_canvas.SetPixel(x, 1, 255, 0, 0)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 255, 0, 0)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 255, 0, 0)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 255, 0, 0)
            offset_canvas.SetPixel(1, y, 255, 0, 0)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 255, 0, 0)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 255, 0, 0)


        for y in range(2):
            for x in range(20):
                offset_canvas.SetPixel(x+21, y+5, 255, 0, 0)
                y = y+1

        for y in range(2):
            for x in range(20):
                offset_canvas.SetPixel(offset_canvas.height + 8 - x, y+5, 255, 0, 0)
                y = y+1


        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)

    def winner(self):
        offset_canvas = self.matrix.CreateFrameCanvas()
        # box of thrphy
        for x in range(0, offset_canvas.width):
            offset_canvas.SetPixel(x, 0, 255, 255, 0)
            offset_canvas.SetPixel(x, 1, 255, 255, 0)
            offset_canvas.SetPixel(x, offset_canvas.height - 1, 255, 255, 0)
            offset_canvas.SetPixel(x, offset_canvas.height - 2, 255, 255, 0)

        for y in range(0, offset_canvas.height):
            offset_canvas.SetPixel(0, y, 255, 255, 0)
            offset_canvas.SetPixel(1, y, 255, 255, 0)
            offset_canvas.SetPixel(offset_canvas.width - 1, y, 255, 255, 0)
            offset_canvas.SetPixel(offset_canvas.width - 2, y, 255, 255, 0)

        for y in range(9):
            for x in range(9):
                offset_canvas.SetPixel(x+28, y+7, 255, 255, 0)

        #base bottom row 1
        for x in range(7):
            offset_canvas.SetPixel(x+29, 16, 255, 255, 0)

        #base bottom row 2
        for x in range(5):
```

```
            offset_canvas.SetPixel(x+30, 17, 255, 255, 0)
            offset_canvas.SetPixel(x+30, 25, 255, 255, 0)

        #base bottom row 3
        for x in range(3):
            offset_canvas.SetPixel(x+31, 18, 255, 255, 0)
            offset_canvas.SetPixel(x+31, 24, 255, 255, 0)

        for x in range(4):
            offset_canvas.SetPixel(x+24, 8, 255, 255, 0)
            offset_canvas.SetPixel(x+37, 8, 255, 255, 0)

        for y in range(4):
            offset_canvas.SetPixel(24, 8+y, 255, 255, 0)
            offset_canvas.SetPixel(41, 8+y, 255, 255, 0)

        for x in range(2):
            offset_canvas.SetPixel(24+x, 11, 255, 255, 0)
            offset_canvas.SetPixel(39+x, 11, 255, 255, 0)
            offset_canvas.SetPixel(37+x, 12, 255, 255, 0)
            offset_canvas.SetPixel(26+x, 12, 255, 255, 0)

        #base col
        for y in range(5):
            offset_canvas.SetPixel(32, y+19, 255, 255, 0)

        offset_canvas = self.matrix.SwapOnVSync(offset_canvas)


    def scoreboard(self, z, offset_canvas):
        t = z//10
        o = z % 10
        if (t) > 0:
            if (t) == 1:
                for y in range(4):
                    #Top part of 1
                    offset_canvas.SetPixel(54, y+5, 255, 255, 0)
                    #bottom part of 1
                    offset_canvas.SetPixel(54, y+8, 255, 255, 0)

            elif (t) == 2:
                for x in range(5):
                    #top part of 2
                    offset_canvas.SetPixel(50+x, 5, 255, 255, 0)
                    #middle part of 2
                    offset_canvas.SetPixel(50+x, 8, 255, 255, 0)
                    #bottom part of 2
                    offset_canvas.SetPixel(50+x, 11, 255, 255, 0)

                for y in range(4):
                    #bottom part of 2
                    offset_canvas.SetPixel(50, y+8, 255, 255, 0)
                    #top part of 2
```

```
            offset_canvas.SetPixel(54, y+5, 255, 255, 0)

    elif (t) == 3:
        for x in range(5):
            #top part of 3
            offset_canvas.SetPixel(50+x, 5, 255, 255, 0)
            #middle part of 3
            offset_canvas.SetPixel(50+x, 8, 255, 255, 0)
            #bottom part of 3
            offset_canvas.SetPixel(50+x, 11, 255, 255, 0)

        for y in range(4):
            #top part of 3
            offset_canvas.SetPixel(54, y+5, 255, 255, 0)
            #bottom part of 3
            offset_canvas.SetPixel(54, y+8, 255, 255, 0)

    elif (t) == 4:
        for x in range(5):
            #middle part of 4
            offset_canvas.SetPixel(50+x, 8, 255, 255, 0)

        for y in range(4):
            #Top part of 4
            offset_canvas.SetPixel(54, y+5, 255, 255, 0)
            #bottom part of 4
            offset_canvas.SetPixel(54, y+8, 255, 255, 0)
            #left part of 4
            offset_canvas.SetPixel(50, y+5, 255, 255, 0)

    elif (t) == 5:
        for x in range(5):
            #top part of 5
            offset_canvas.SetPixel(50+x, 5, 255, 255, 0)
            #middle part of 5
            offset_canvas.SetPixel(50+x, 8, 255, 255, 0)
            #bottom part of 5
            offset_canvas.SetPixel(50+x, 11, 255, 255, 0)

        for y in range(4):
            #bottom part of 5
            offset_canvas.SetPixel(54, y+8, 255, 255, 0)
            #left part of 5
            offset_canvas.SetPixel(50, y+5, 255, 255, 0)

    elif (t) == 6:
        for x in range(5):
            #top part of 6
            offset_canvas.SetPixel(50+x, 5, 255, 255, 0)
            #middle part of 6
            offset_canvas.SetPixel(50+x, 8, 255, 255, 0)
            #bottom part of 6
            offset_canvas.SetPixel(50+x, 11, 255, 255, 0)
```

```
    for y in range(4):
        #bottom part of 6
        offset_canvas.SetPixel(54, y+8, 255, 255, 0)
        #left top of 6
        offset_canvas.SetPixel(50, y+5, 255, 255, 0)
        #left bottom of 6
        offset_canvas.SetPixel(50, y+8, 255, 255, 0)

elif (t) == 7:
    for x in range(5):
        #top part of 7
        offset_canvas.SetPixel(50+x, 5, 255, 255, 0)

    for y in range(4):
        #Top part of 7
        offset_canvas.SetPixel(54, y+5, 255, 255, 0)
        #bottom part of 7
        offset_canvas.SetPixel(54, y+8, 255, 255, 0)

elif (t) == 8:
    for x in range(5):
        #top part of 8
        offset_canvas.SetPixel(50+x, 5, 255, 255, 0)
        #middle part of 8
        offset_canvas.SetPixel(50+x, 8, 255, 255, 0)
        #bottom part of 8
        offset_canvas.SetPixel(50+x, 11, 255, 255, 0)

    for y in range(4):
        #left top of 8
        offset_canvas.SetPixel(50, y+5, 255, 255, 0)
        #left bottom of 8
        offset_canvas.SetPixel(50, y+8, 255, 255, 0)
        #top part of 8
        offset_canvas.SetPixel(54, y+5, 255, 255, 0)
        #bottom part of 8
        offset_canvas.SetPixel(54, y+8, 255, 255, 0)


elif (t) == 9:
    for x in range(5):
        #top part of 9
        offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
        #middle part of 9
        offset_canvas.SetPixel(56+x, 8, 255, 255, 0)

    for y in range(4):
        #left top of 9
        offset_canvas.SetPixel(50, y+5, 255, 255, 0)
        #top part of 9
        offset_canvas.SetPixel(54, y+5, 255, 255, 0)
        #bottom part of 9
        offset_canvas.SetPixel(54, y+8, 255, 255, 0)
```

```
if (o) == 0:
    for x in range(5):
        #top part of 0
        offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
        #bottom part of 0
        offset_canvas.SetPixel(56+x, 11, 255, 255, 0)

    for y in range(4):
        #left top of 0
        offset_canvas.SetPixel(56, y+5, 255, 255, 0)
        #left bottom of 0
        offset_canvas.SetPixel(56, y+8, 255, 255, 0)
        #top part of 0
        offset_canvas.SetPixel(60, y+5, 255, 255, 0)
        #bottom part of 0
        offset_canvas.SetPixel(60, y+8, 255, 255, 0)

elif (o) == 1:
    for y in range(4):
        #Top part of 1
        offset_canvas.SetPixel(60, y+5, 255, 255, 0)
        #bottom part of 1
        offset_canvas.SetPixel(60, y+8, 255, 255, 0)

elif (o) == 2:
    for x in range(5):
        #top part of 2
        offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
        #middle part of 2
        offset_canvas.SetPixel(56+x, 8, 255, 255, 0)
        #bottom part of 2
        offset_canvas.SetPixel(56+x, 11, 255, 255, 0)

    for y in range(4):
        #bottom part of 2
        offset_canvas.SetPixel(56, y+8, 255, 255, 0)
        #top part of 2
        offset_canvas.SetPixel(60, y+5, 255, 255, 0)

elif (o) == 3:
    for x in range(5):
        #top part of 3
        offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
        #middle part of 3
```

```
            offset_canvas.SetPixel(56+x, 8, 255, 255, 0)
            #bottom part of 3
            offset_canvas.SetPixel(56+x, 11, 255, 255, 0)

        for y in range(4):
            #top part of 3
            offset_canvas.SetPixel(60, y+5, 255, 255, 0)
            #bottom part of 3
            offset_canvas.SetPixel(60, y+8, 255, 255, 0)

    elif (o) == 4:
        for x in range(5):
            #middle part of 4
            offset_canvas.SetPixel(56+x, 8, 255, 255, 0)

        for y in range(4):
            #Top part of 4
            offset_canvas.SetPixel(60, y+5, 255, 255, 0)
            #bottom part of 4
            offset_canvas.SetPixel(60, y+8, 255, 255, 0)
            #left part of 4
            offset_canvas.SetPixel(56, y+5, 255, 255, 0)

    elif (o) == 5:
        for x in range(5):
            #top part of 5
            offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
            #middle part of 5
            offset_canvas.SetPixel(56+x, 8, 255, 255, 0)
            #bottom part of 5
            offset_canvas.SetPixel(56+x, 11, 255, 255, 0)

        for y in range(4):
            #bottom part of 5
            offset_canvas.SetPixel(60, y+8, 255, 255, 0)
            #left part of 5
            offset_canvas.SetPixel(56, y+5, 255, 255, 0)

    elif (o) == 6:
        for x in range(5):
            #top part of 6
            offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
            #middle part of 6
            offset_canvas.SetPixel(56+x, 8, 255, 255, 0)
            #bottom part of 6
            offset_canvas.SetPixel(56+x, 11, 255, 255, 0)
        for y in range(4):
            #bottom part of 6
            offset_canvas.SetPixel(60, y+8, 255, 255, 0)
            #left top of 6
            offset_canvas.SetPixel(56, y+5, 255, 255, 0)
            #left bottom of 6
            offset_canvas.SetPixel(56, y+8, 255, 255, 0)
```

```
        elif (o) == 7:
            for x in range(5):
                #top part of 7
                offset_canvas.SetPixel(56+x, 5, 255, 255, 0)

            for y in range(4):
                #Top part of 7
                offset_canvas.SetPixel(60, y+5, 255, 255, 0)
                #bottom part of 7
                offset_canvas.SetPixel(60, y+8, 255, 255, 0)

        elif (o) == 8:
            for x in range(5):
                #top part of 8
                offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
                #middle part of 8
                offset_canvas.SetPixel(56+x, 8, 255, 255, 0)
                #bottom part of 8
                offset_canvas.SetPixel(56+x, 11, 255, 255, 0)

            for y in range(4):
                #left top of 8
                offset_canvas.SetPixel(56, y+5, 255, 255, 0)
                #left bottom of 8
                offset_canvas.SetPixel(56, y+8, 255, 255, 0)
                #top part of 8
                offset_canvas.SetPixel(60, y+5, 255, 255, 0)
                #bottom part of 8
                offset_canvas.SetPixel(60, y+8, 255, 255, 0)


        elif (o) == 9:
            for x in range(5):
                #top part of 9
                offset_canvas.SetPixel(56+x, 5, 255, 255, 0)
                #middle part of 9
                offset_canvas.SetPixel(56+x, 8, 255, 255, 0)

            for y in range(4):
                #left top of 9
                offset_canvas.SetPixel(56, y+5, 255, 255, 0)
                #top part of 9
                offset_canvas.SetPixel(60, y+5, 255, 255, 0)
                #bottom part of 9
                offset_canvas.SetPixel(60, y+8, 255, 255, 0)


# Main function
if __name__ == "__main__":
    simple_square = SimpleSquare()
    if (not simple_square.process()):
        simple_square.print_help()
```